

LAPORAN PRAKTIKUM
PERANCANGAN DAN PEMROGRAMAN WEB

MODUL 13
(Node.js: Pengenalan)



Oleh:

(Rifqi M Ramdani)

(2311104044)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2025

BAB I

PENDAHULUAN

A. Dasar Teori

Node.js adalah runtime environment JavaScript yang bersifat open-source dan cross-platform, memungkinkan eksekusi kode JavaScript di sisi server. Berbeda dengan JavaScript tradisional yang berjalan di browser, Node.js menggunakan arsitektur event-driven dan non-blocking I/O yang membuatnya efisien untuk aplikasi real-time dan I/O intensive. Express.js berfungsi sebagai framework web minimalis yang menyederhanakan pembuatan aplikasi dan API dengan menyediakan routing, middleware, dan HTTP utility. MySQL merupakan sistem manajemen database relasional (RDBMS) yang menggunakan Structured Query Language (SQL) untuk mengelola data dalam bentuk tabel terstruktur. RESTful API (Representational State Transfer) adalah arsitektur gaya arsitektur yang menggunakan protokol HTTP untuk operasi CRUD (Create, Read, Update, Delete) dengan pendekatan stateless antara klien dan server.

B. Tujuan

Tujuan utama praktikum ini adalah mengimplementasikan aplikasi web pengelolaan data mahasiswa menggunakan teknologi Node.js, Express.js, dan MySQL. Secara spesifik, praktikum bertujuan untuk: pertama, memahami konsep pembuatan RESTful API dengan Node.js dan Express.js; kedua, menguasai teknik koneksi dan operasi database MySQL dari aplikasi Node.js; ketiga, mengimplementasikan operasi CRUD lengkap (Create, Read, Update, Delete) pada data mahasiswa; keempat, membangun integrasi antara frontend (HTML, CSS, JavaScript) dengan backend API; dan kelima, menerapkan pola arsitektur MVC (Model-View-Controller) secara sederhana melalui pemisahan concerns antara layer database, business logic, dan presentation layer

BAB II

HASIL

A. UNGUIDED (Tugas Mandiri)

1. Mahasiswa diminta untuk membangun sebuah **aplikasi web sederhana** untuk pengelolaan data mahasiswa berbasis Node.js, Express.js, dan MySQL.

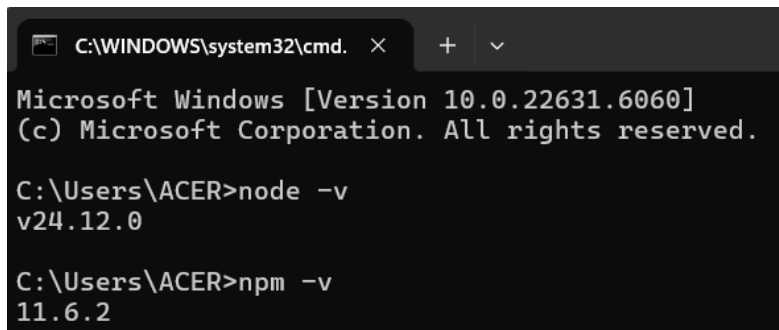
Aplikasi harus menyediakan **RESTful API** serta dapat diakses melalui browser menggunakan halaman web sederhana (HTML dan JavaScript).

Mahasiswa diperbolehkan menggunakan proyek yang telah dibuat di atas atau membuat proyek baru.

Jawaban:

Langkah Pertama Instal Node.js

Lalu verifikasi instalasi

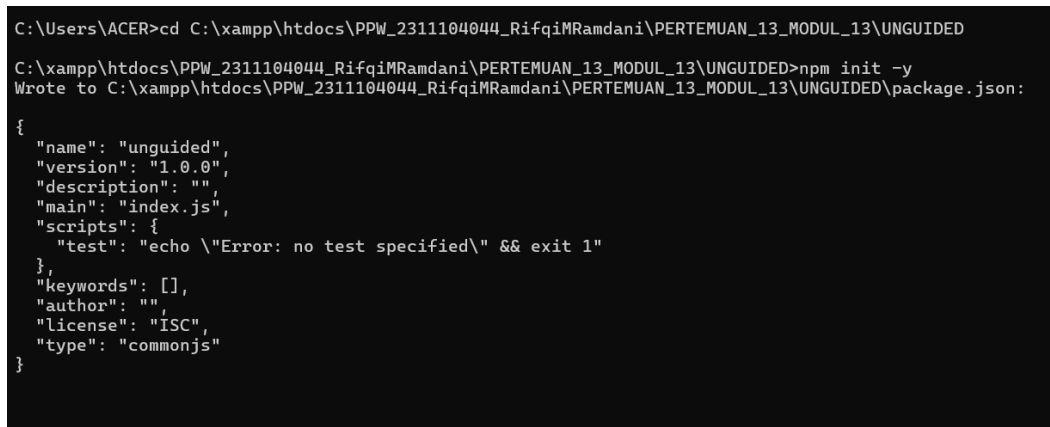


```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [Version 10.0.22631.6060]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER>node -v
v24.12.0

C:\Users\ACER>npm -v
11.6.2
```

Inisialisasi Proyek Node.js



```
C:\Users\ACER>cd C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_13_MODUL_13\UNGUIDED
C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_13_MODUL_13\UNGUIDED>npm init -y
Wrote to C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_13_MODUL_13\UNGUIDED\package.json:

{
  "name": "unguided",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Selanjutnya Install Dependencies

```
C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_13_MODUL_13\UNGUIDED>npm install express mysql
added 75 packages, and audited 76 packages in 5s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

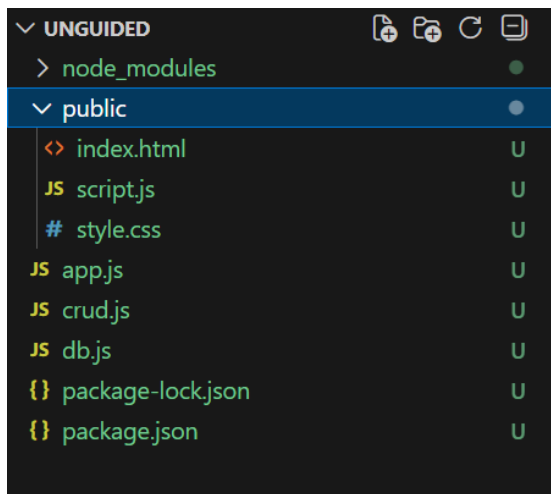
C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_13_MODUL_13\UNGUIDED>npm install --save-dev nodemon
added 27 packages, and audited 103 packages in 3s

26 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\xampp\htdocs\PPW_2311104044_RifqiMRamdani\PERTEMUAN_13_MODUL_13\UNGUIDED>
```

Struktur Folder



Source Code:

Buat File db.js

```
const mysql = require('mysql');

// Koneksi ke database MySQL (XAMPP default)
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '', // Password kosong untuk XAMPP default
  database: 'akademik' // Sesuaikan dengan nama database Anda
});

// Tes koneksi
connection.connect((err) => {
  if (err) {
    console.error('❌ ERROR Koneksi Database:', err.message);
    console.log('Pastikan:');
    console.log('1. XAMPP MySQL sedang running');
    console.log('2. Database "akademik" sudah dibuat');
    console.log('3. Username: root, Password: (kosong)');
    return;
  }
  console.log('✅ SUCCESS: Terhubung ke MySQL Database!');
  console.log('📄 Database:', connection.config.database);
  console.log('🔗 Connection ID:', connection.threadId);
});

module.exports = connection;
```

menghubungkan aplikasi ke database MySQL. Ia menggunakan konfigurasi localhost dengan user root dan database akademik. Jika koneksi berhasil, ia menampilkan pesan sukses; jika gagal, menampilkan error dan cara memperbaikinya.

Lalu crud.js

```
const connection = require('./db');

const mahasiswaModel = {
  // CREATE - Tambah data baru
  create: (data, callback) => {
    const sql = "INSERT INTO mahasiswa SET ?";
    connection.query(sql, data, (err, result) => {
      callback(err, result);
    });
  },

  // READ ALL - Ambil semua data
  findAll: (callback) => {
    const sql = "SELECT * FROM mahasiswa ORDER BY id DESC";
    connection.query(sql, (err, results) => {
      callback(err, results);
    });
  },

  // READ ONE - Ambil data by ID
  findById: (id, callback) => {
    const sql = "SELECT * FROM mahasiswa WHERE id = ?";
    connection.query(sql, [id], (err, result) => {
      callback(err, result);
    });
  },

  // UPDATE - Update data by ID
  update: (id, data, callback) => {
    const sql = "UPDATE mahasiswa SET ? WHERE id = ?";
    connection.query(sql, [data, id], (err, result) => {
      callback(err, result);
    });
  },

  // DELETE - Hapus data by ID
  delete: (id, callback) => {
    const sql = "DELETE FROM mahasiswa WHERE id = ?";
    connection.query(sql, [id], (err, result) => {
      callback(err, result);
    });
  }
};

module.exports = mahasiswaModel;
```

mengelola operasi database untuk data mahasiswa. Ia berisi fungsi untuk menambah, melihat semua, melihat berdasarkan ID, mengubah, dan menghapus

data dari tabel mahasiswa. Semua fungsi menggunakan callback untuk menangani hasil atau error.

App.js

```

const express = require('express');
const mahasiswa = require('./crud');
const path = require('path');

const app = express();
const PORT = 3000;

// ===== MIDDLEWARE =====
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// ===== ROUTES API =====

// GET: /api/mahasiswa - Ambil semua data
app.get('/api/mahasiswa', (req, res) => {
  mahasiswa.findAll((err, data) => {
    if (err) {
      return res.status(500).json({
        success: false,
        message: 'Gagal mengambil data',
        error: err.message
      });
    }
    res.json({
      success: true,
      count: data.length,
      data: data
    });
  });
});

// POST: /api/mahasiswa - Tambah data baru
app.post('/api/mahasiswa', (req, res) => {
  const { nama, nim, jurusan, email } = req.body;

  // Validasi input
  if (!nama || !nim || !jurusan || !email) {
    return res.status(400).json({
      success: false,
      message: 'Semua field harus diisi!'
    });
  }

  const data = { nama, nim, jurusan, email };
  mahasiswa.create(data, (err, result) => {
    if (err) {
      return res.status(500).json({
        success: false,
        message: 'Gagal menambah data',
        error: err.message
      });
    }
    res.status(201).json({
      success: true,
      message: 'Data berhasil ditambahkan',
      id: result.insertId
    });
  });
});

// PUT: /api/mahasiswa/:id - Update data
app.put('/api/mahasiswa/:id', (req, res) => {
  const id = req.params.id;
  const { nama, nim, jurusan, email } = req.body;

  if (!nama || !nim || !jurusan || !email) {
    return res.status(400).json({
      success: false,
      message: 'Semua field harus diisi!'
    });
  }

  const data = { nama, nim, jurusan, email };
  mahasiswa.update(id, data, (err, result) => {
    if (err) {
      return res.status(500).json({
        success: false,
        message: 'Gagal mengupdate data',
        error: err.message
      });
    }
    res.json({
      success: true,
      message: 'Data berhasil diupdate',
      affectedRows: result.affectedRows
    });
  });
});

// DELETE: /api/mahasiswa/:id - Hapus data
app.delete('/api/mahasiswa/:id', (req, res) => {
  const id = req.params.id;
  mahasiswa.delete(id, (err, result) => {
    if (err) {
      return res.status(500).json({
        success: false,
        message: 'Gagal menghapus data',
        error: err.message
      });
    }
    res.json({
      success: true,
      message: 'Data berhasil dihapus',
      affectedRows: result.affectedRows
    });
  });
});

// ===== ROUTES HALAMAN =====
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'public', 'index.html'));
});

// ===== START SERVER =====
app.listen(PORT, () => {
  console.log('🔥 SERVER MODE JS BERJALAN');
  console.log('🔥 Port: http://localhost:${PORT}');
  console.log('🔥 Public: http://localhost:${PORT}/public');
  console.log('🔥 API: http://localhost:${PORT}/api/mahasiswa');
  console.log('🔥 Tekan Ctrl+C untuk stop server');
});

```

server utama yang menggunakan Express.js. Ia membuat API dengan 4 endpoint: POST untuk tambah data, GET untuk lihat data, PUT untuk ubah data, dan DELETE untuk hapus data. Server berjalan di port 3000 dan juga menyajikan file frontend dari folder public.

Index.html

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Modul 11 - Manajemen Data Mahasiswa</title>
  <link rel="stylesheet" href="style.css">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
</head>
<body>
  <div class="container">
    <!-- Header -->
    <header class="header">
      <h1><div class="fas fa-graduation-cap"></div> MODUL 11 - Node.js</h1>
      <p class="subitle">Aplikasi Manajemen Data Mahasiswa | NPM: 231104044 | Nama: Rifqi M Randani</p>
    </header>
    <div class="server-info">
      <span id="serverStatus"><div class="fas fa-circle"></div> Server Offline</span>
      <button onclick="checkServer()">Periksa Server</button>
    </div>
    </header>
    <!-- Form Input Data -->
    <section class="card">
      <h2><div class="fas fa-user-plus"></div> Form Input Data Mahasiswa</h2>
      <form id="formData" onsubmit="handleSubmit(event)">
        <div class="form-row">
          <div class="form-group">
            <label for="nama"><div class="fas fa-user"></div> Nama Lengkap</label>
            <input type="text" id="nama" placeholder="Contoh: Rifqi M Randani" required>
          </div>
          <div class="form-group">
            <label for="nim"><div class="fas fa-id-card"></div> NIM</label>
            <input type="text" id="nim" placeholder="Contoh: 231104044" required>
          </div>
        </div>
        <div class="form-row">
          <div class="form-group">
            <label for="jurusan"><div class="fas fa-book"></div> Jurusan</label>
            <input type="text" id="jurusan" placeholder="Contoh: Teknik Informatika" required>
          </div>
          <div class="form-group">
            <label for="email"><div class="fas fa-envelope"></div> Email</label>
            <input type="email" id="email" placeholder="Contoh: rifq@gmail.com" required>
          </div>
        </div>
        <div class="form-actions">
          <button type="submit" class="btn-primary"><div class="fas fa-save"></div> Simpan Data</button>
          <button type="button" onclick="resetForm()" class="btn-secondary"><div class="fas fa-redo"></div> Reset Form</button>
        </div>
      </form>
    </section>
    <!-- Daftar Data -->
    <section class="card">
      <div class="card-header">
        <h2><div class="fas fa-list"></div> Daftar Mahasiswa</h2>
        <div class="controls">
          <button onclick="loadData()" class="btn-refresh"><div class="fas fa-sync-alt"></div> Refresh Data</button>
          <span id="totalData">Memuat data...</span>
        </div>
      </div>
      <div class="table-container">
        <table id="tabelMahasiswa">
          <thead>
            <tr>
              <th><div class="fas fa-id-card"></div></th>
              <th><div class="fas fa-user"></div> Nama</th>
              <th><div class="fas fa-id-card"></div> NIM</th>
              <th><div class="fas fa-book"></div> Jurusan</th>
              <th><div class="fas fa-envelope"></div> Email</th>
              <th><div class="fas fa-graduation-cap"></div> Aksi</th>
            </tr>
          </thead>
          <tbody id="dataBody">
            <tr>
              <td colspan="6" class="loading">Memuat data...</td>
            </tr>
          </tbody>
        </table>
      </div>
    </section>
    <!-- Footer -->
    <footer class="footer">
      <p>© 2024 - Praktikum Pemrograman Web - Node.js Module 13</p>
      <p>Dibuat dengan Node.js, Express.js, MySQL, HTML, CSS, JavaScript</p>
    </footer>
  </div>
  <!-- Modal Edit -->
  <div id="editModal" class="modal">
    <div class="modal-content">
      <div class="modal-header">
        <h3><div class="fas fa-edit"></div> Edit Data Mahasiswa</h3>
        <span class="close" onclick="closeModal()"><div class="fas fa-times"></div></span>
      </div>
      <form id="formEdit" onsubmit="updateData(event)">
        <input type="hidden" id="editId">
        <div class="form-group">
          <label>Nama</label>
          <input type="text" id="editNama" required>
        </div>
        <div class="form-group">
          <label>NIM</label>
          <input type="text" id="editNim" required>
        </div>
        <div class="form-group">
          <label>Jurusan</label>
          <input type="text" id="editJurusan" required>
        </div>
        <div class="form-group">
          <label>Email</label>
          <input type="email" id="editEmail" required>
        </div>
        <div class="modal-actions">
          <button type="submit" class="btn-primary">Update Data</button>
          <button type="button" onclick="closeModal()" class="btn-secondary">Batal</button>
        </div>
      </form>
    </div>
  </div>
  <script src="script.js"></script>
</body>
</html>

```

halaman web utama dengan form input data mahasiswa dan tabel untuk menampilkan data. Ia memiliki tampilan yang menarik dengan CSS dan logika JavaScript untuk berinteraksi dengan API.

style.css

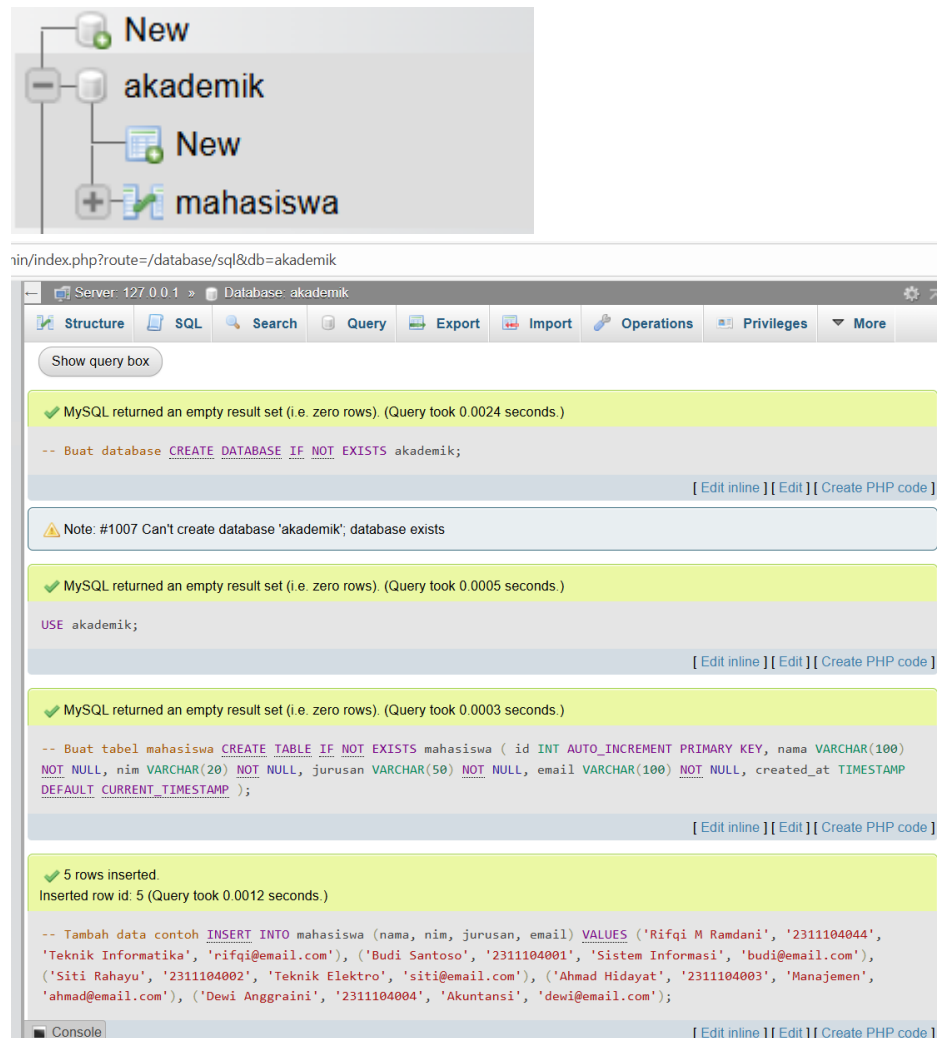
[illegible]

mengatur tampilan visual aplikasi: layout, warna, font, dan responsivitas halaman web untuk pengalaman pengguna yang baik.

Script.js

menghubungkan frontend dengan backend API. Ia menangani pengiriman data dari form, menampilkan data di tabel, serta mengelola edit dan hapus data melalui permintaan HTTP ke server.

Buat database



The screenshot shows a database management interface with a tree view on the left containing 'New', 'akademik', 'New', and 'mahasiswa'. The main window displays the SQL console for the 'akademik' database. The console shows the execution of three SQL queries: creating the database, selecting the database, and creating a table named 'mahasiswa' with 5 rows inserted. The interface includes a toolbar with options like Structure, SQL, Search, Query, Export, Import, Operations, Privileges, and More. The console output shows success messages for each query, including the number of rows inserted (5 rows).

```
-- Buat database CREATE DATABASE IF NOT EXISTS akademik;
```

```
USE akademik;
```

```
-- Buat tabel mahasiswa CREATE TABLE IF NOT EXISTS mahasiswa ( id INT AUTO_INCREMENT PRIMARY KEY, nama VARCHAR(100) NOT NULL, nim VARCHAR(20) NOT NULL, jurusan VARCHAR(50) NOT NULL, email VARCHAR(100) NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

```
-- Tambah data contoh INSERT INTO mahasiswa (nama, nim, jurusan, email) VALUES ('Rifqi M Ramdani', '2311104044', 'Teknik Informatika', 'rifqi@email.com'), ('Budi Santoso', '2311104001', 'Sistem Informasi', 'budi@email.com'), ('Siti Rahayu', '2311104002', 'Teknik Elektro', 'siti@email.com'), ('Ahmad Hidayat', '2311104003', 'Manajemen', 'ahmad@email.com'), ('Dewi Anggraini', '2311104004', 'Akuntansi', 'dewi@email.com');
```

Output:

The screenshot displays a web application titled "MODUL 13 - Node.js" with the subtitle "Aplikasi Manajemen Data Mahasiswa | NPM: 2311104044 | Nama: Rifqi M Ramdani". The interface is divided into two main sections: a form for adding new student data and a table for managing existing student data.

Form Input Data Mahasiswa

The form contains the following fields:

- Nama Lengkap: Hansohee
- NIM: 2311104044
- Jurusan: RPL
- Email: 2311104044@ittelkom-pwt.ac.id

Buttons: Simpan Data, Reset Form

Daftar Mahasiswa

Refresh Data button. Total: 6 data

ID	Nama	NIM	Jurusan	Email	Aksi
6	Hansohee	2311104044	RPL	2311104044@ittelkom-pwt.ac.id	Edit Hapus
5	Dewi Anggraini	2311104004	Akuntansi	dewi@email.com	Edit Hapus
4	Ahmad Hidayat	2311104003	Manajemen	ahmad@email.com	Edit Hapus
3	Siti Rahayu	2311104002	Teknik Elektro	siti@email.com	Edit Hapus
2	Budi Santoso	2311104001	Sistem Informasi	budi@email.com	Edit Hapus
1	Rifqi M Ramdani	2311104044	Teknik Informatika	rifqi@email.com	Edit Hapus

© 2024 - Praktikum Pemrograman Web - Node.js Module 13
Dibuat dengan Node.js, Express.js, MySQL, HTML, CSS, JavaScript

Server berjalan dengan baik!

Penjelasan:

J berfungsi sebagai sistem manajemen data mahasiswa berbasis web yang terdiri dari frontend dan backend terintegrasi. Cara kerjanya dimulai dari server Node.js (app.js) yang menerima permintaan HTTP dari browser, kemudian memprosesnya melalui API endpoints yang sesuai, selanjutnya mengakses database MySQL melalui koneksi yang diatur di db.js dan operasi CRUD yang didefinisikan di crud.js. Frontend yang terdiri dari index.html, style.css, dan script.js menyediakan antarmuka pengguna yang memungkinkan input, tampilan, dan manipulasi data secara real-time. Aliran data terjadi secara sirkuler: pengguna berinteraksi dengan antarmuka web, frontend mengirim permintaan ke API backend, backend memproses dan menyimpan/ mengambil data dari database, kemudian mengembalikan respons ke frontend untuk ditampilkan kepada pengguna, menciptakan aplikasi CRUD lengkap yang bekerja sepenuhnya di browser dengan dukungan server Node.js dan database MySQL.

BAB III

PENUTUP

A. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Node.js bersama Express.js berhasil membentuk platform yang efektif untuk pengembangan aplikasi web server-side dengan JavaScript. Implementasi RESTful API terbukti memberikan fleksibilitas dalam komunikasi antara frontend dan backend, sementara koneksi dengan MySQL database bekerja dengan optimal untuk penyimpanan data terstruktur. Aplikasi yang dibangun telah memenuhi semua fungsi CRUD dasar dengan antarmuka pengguna yang responsif dan intuitif. Penggunaan arsitektur modular (db.js, crud.js, app.js) meningkatkan maintainability dan scalability kode. Secara keseluruhan, praktikum ini berhasil mendemonstrasikan kemampuan membangun aplikasi web full-stack dengan teknologi modern yang ringan dan performa tinggi.